# Provider Script Documentation

Developer manual for writing Streamora provider scripts

## Overview

Providers are Python scripts that integrate external streaming services into Streamora. Each script implements a simple CLI protocol to import channels, fetch stream manifests, and acquire DRM keys.

### How It Works

Streamora calls your script as a subprocess, passing parameters as command-line arguments. Your script does its work (API calls, authentication, etc.) and prints a JSON result to `stdout`. Log messages go to `stderr` and are shown in the Streamora terminal UI in real time.

### Execution Flow

```
User clicks "Sync" on a provider
    |
    v
Step 1: python3 script.py action=channels user=... password=...
    |   -> Returns list of channels (JSON to stdout)
    v
Step 2: For EACH channel:
        python3 script.py action=manifest id=ch_123 user=... password=...
        |   -> Returns stream URL, DRM info (JSON to stdout)
        v
Step 3: If DRM and no keys yet:
        Built-in Widevine CDM tried first
        |   -> If that fails:
        v   python3 script.py action=cdm id=ch_123 user=... password=...
            -> Returns KID:KEY pairs (JSON to stdout)
```

### Interactive Mode

```
User clicks "Interactive" button on a provider
    |
    v
python3 -u script.py action=interactive user=... password=...
    |   -> Script runs with stdin connected
    |   -> stdout + stderr streamed to terminal in real time
    |   -> User types input in terminal -> sent to stdin
    |   -> Script can use input() to read user input
    v
Session ends when script exits (or 10 min timeout)
```

Use interactive mode for scripts that need user input (OTP, subscriber ID, menu choices). Unlike sync/refresh, the script runs as a single long-lived process with bidirectional I/O.

### Key Concepts

**Channels** are matched by their `id` field. Re-syncing updates existing channels, never creates duplicates.

**Credentials** (email, password, API keys) are stored in Streamora and passed as CLI arguments.

**Config** (proxy, CDN, region) is also passed as CLI arguments, separate from credentials.

**Scripts are stateless —** each invocation is independent. Use file-based caching for tokens if needed.

**Timeout** is 30 minutes per sync action. Interactive sessions timeout after 10 minutes of inactivity.

## Script Interface

### CLI Convention

Scripts are invoked as:

```
python3 script.py action=X id=Y key=value ...
```

All parameters are passed as `key=value` pairs in `sys.argv`. The working directory is set to the folder containing the script.

### Standard Parameters

| Parameter | Source | Description |
|---|---|---|
| `action` | System | `channels`, `manifest`, or `cdm` |
| `id` | System | Channel ID (for manifest/cdm actions) |
| *credentials* | Provider config | Each JSON key becomes a CLI arg (e.g. `user=john`) |
| *config* | Provider config | Each JSON key becomes a CLI arg (e.g. `proxy=http://...`) |

### Argument Parsing

The standard pattern to parse arguments:

```
import sys

def parse_arg(args, key):
    """Extract value for key=value from argv."""
    for arg in args:
        if arg.startswith(key + '='):
            return arg.split('=', 1)[1]
    return None

action = parse_arg(sys.argv, 'action')
channel_id = parse_arg(sys.argv, 'id')
username = parse_arg(sys.argv, 'user')
password = parse_arg(sys.argv, 'password')
```

### Output Rules

- `stdout` — JSON result only. Must be valid JSON. One `print(json.dumps(...))` call.
- `stderr` — Log/progress messages. Shown in Streamora terminal UI. Use `print(..., file=sys.stderr, flush=True)`.
- Exit code `0` = success, non-zero = error.
- Include `"error": "message"` in JSON to report errors without crashing.

## Actions & JSON Schemas

**action=channels**

Returns the full list of channels from the provider. Called once per sync.

REQUEST

```
python3 script.py action=channels user=john@email.com password=secret
```

RESPONSE SCHEMA

```
{
  "channels": [
    {
      "id": "unique_channel_id",     // REQUIRED - unique within this provider
      "name": "ESPN HD",              // REQUIRED - display name
      "source_url": "",               // Stream URL (empty if manifest provides it)
      "source_type": "dash",          // hls | dash | direct
      "drm_type": "widevine",         // none | widevine | clearkey | playready
      "group": "Sports",              // Profile name — auto-created on first sync
      "logo": "https://...",          // Channel logo URL
      "epg_id": "espn.us"            // EPG guide mapping ID
    }
  ],
  "profile_defaults": {              // OPTIONAL — auto-configure profiles
    "Sports": {                      // Keys match "group" names above
      "running_mode": "internal_remux",
      "output_format": "hls",
      "manifest_refresh_min": 5,
      "proxy_from_provider": true,
      "shared_headers": 1
    }
  }
}
```

FIELD REFERENCE

| Field | Type | Required | Description |
|---|---|---|---|
| `id` | string | required | Unique channel identifier within this provider |
| `name` | string | required | Display name for the channel |
| `source_url` | string | optional | Direct stream URL. Leave empty if `action=manifest` provides it |
| `source_type` | string | optional | `hls`, `dash`, `direct`, or `smooth` |
| `drm_type` | string | optional | `none`, `widevine`, `clearkey`, `playready` |
| `group` | string | optional | Profile/group name. Overrides provider-level profile |
| `logo` | string | optional | URL to channel logo image |
| `epg_id` | string | optional | EPG channel ID for program guide matching |

**action=manifest**

Returns the stream manifest URL and DRM details for a specific channel. Called per channel during sync.

REQUEST

```
python3 script.py action=manifest id=unique_channel_id user=john@email.com password=secret
```

RESPONSE SCHEMA

```
{
  "manifest_url": "https://cdn.example.com/stream.mpd",
  "manifest_headers": {
    "Authorization": "Bearer token123",
    "Origin": "https://example.com"
```

```
  },
  "media_headers": {},
  "license_url": "https://drm.example.com/license",
  "license_headers": {
    "Origin": "https://example.com",
    "X-Token": "abc"
  },
  "pssh": "AAAAOHBzc2gAAAAA7e+LqXnWSs6jyCfc1R0h7Q...",
  "drm_type": "widevine",
  "source_type": "dash",
  "keys": "",
  "error": ""
}
```

FIELD REFERENCE

| Field | Type | Description |
|---|---|---|
| `manifest_url` | string | HLS (.m3u8) or DASH (.mpd) manifest URL |
| `manifest_headers` | object | HTTP headers for manifest request |
| `media_headers` | object | HTTP headers for media/segment requests. If empty and shared_headers=1, manifest_headers are used for both |
| `license_url` | string | DRM license server URL |
| `license_headers` | object | HTTP headers for license request |
| `pssh` | string | Base64 PSSH box (extracted from MPD or init segment) |
| `drm_type` | string | `none`, `widevine`, `clearkey`, `playready` |
| `source_type` | string | `hls`, `dash`, `direct`, `smooth` |
| `keys` | string | Direct ClearKey pairs (`kid:key`). If provided, CDM step is skipped |
| `error` | string | Error message (empty on success) |

**Tip:** If you can return `keys` directly (e.g. ClearKey), do so. This skips the CDM step entirely and is faster.

**action=cdm**

Acquires DRM decryption keys. This is a **fallback** — only called when the built-in Widevine CDM fails. Most scripts don't need this.

REQUEST

```
python3 script.py action=cdm id=unique_channel_id user=john@email.com password=secret
```

RESPONSE SCHEMA

```
{
  "keys": "aabb1122aabb1122:1122aabb1122aabb\nccdd3344ccdd3344:3344ccdd3344ccdd",
  "error": ""
}
```

KEY FORMAT

Each key is `KID:KEY` where both are lowercase hex strings

Multiple keys separated by newlines (`\n`)

Example: `aabbccdd11223344:11223344aabbccdd`

**action=interactive**

Interactive terminal mode. The script runs with **stdin connected**, allowing the user to provide input (OTP, subscriber ID, menu choices) via the admin UI. Output from both `stdout` and `stderr` is streamed to the terminal in real time.

HOW IT WORKS

Script is launched with `python3 -u script.py action=interactive ...` (unbuffered mode)

Both `stdout` and `stderr` are streamed line-by-line to the admin terminal

The user types input in the terminal input bar — each submission writes a line to `stdin`

Use standard `input()` to read user input, and `print()` to display output

Session timeout: 10 minutes of inactivity

EXAMPLE

```
if action == "interactive":
    print("=== Interactive Login ===")
    print("Enter subscriber ID:")
    sub_id = input().strip()
    print(f"Subscriber: {sub_id}")

    # Request OTP
    send_otp(sub_id)
    print("OTP sent to your phone. Enter OTP:")
    otp = input().strip()

    token = verify_otp(sub_id, otp)
    if not token:
        print("ERROR: Invalid OTP")
        sys.exit(1)

    print(f"Login successful! Token: {token[:20]}...")
    print("")
    print("Options:")
    print("  1 - Dump all channels")
    print("  2 - Dump specific channel")
    print("  3 - Refresh token only")
    choice = input().strip()

    if choice == "1":
        channels = get_all_channels(token)
        print(f"Found {len(channels)} channels")
        # Save/process channels...
    elif choice == "2":
        print("Enter channel ID:")
        ch_id = input().strip()
        # Process single channel...
    elif choice == "3":
        save_token(token)
        print("Token saved successfully")
    else:
        print("Invalid choice")

    print("Done!")
    sys.exit(0)
```

TIPS

- Always use `flush=True` with `print()` or run with `python3 -u` (Streamora does this automatically)
- The `input()` prompt text is ignored — print your prompt with `print()` instead
- User input is shown in the terminal as blue `> text` lines
- Exit cleanly with `sys.exit(0)` for success or `sys.exit(1)` for error
- Unlike other actions, `stdout` is NOT parsed as JSON — print freely

## Profile Auto-Configuration

Scripts can auto-configure stream profiles when channels are first synced. Return `profile_defaults` in the `action=channels` output, keyed by group name. These defaults are applied when profiles are **created** — existing profiles are not overwritten unless "Update profile settings" is checked in the sync modal.

### How It Works

Script returns channels with `"group": "DAZN Linear"`

Streamora creates a profile named "DAZN Linear" (if it doesn't exist)

If `profile_defaults["DAZN Linear"]` exists, those settings are applied

All channels in that group inherit the profile's settings

### All Fields

| Key | Type | Default | Description |
|---|---|---|---|
| `running_mode` | string | `""` | Pipeline mode: `internal_remux` (DASH, recommended), `ffmpeg_remux` (HLS), `ffmpeg_transcode`, `direct` |
| `output_format` | string | `""` | Output format: `hls` (recommended) or `mpegts` |
| `manifest_refresh_min` | int | `0` | Minutes between CDN token refreshes during playback. Set 5-8 for expiring CDN tokens. 0 = disabled |
| `skip_prestart_refresh` | int | `0` | 1 = skip calling script before channel start. Use for non-expiring sources |
| `proxy_from_provider` | bool | `false` | `true` = copy the proxy URL from provider Config JSON to the profile. Essential for geo-restricted CDNs |
| `shared_headers` | int | `0` | 1 = CDN auth headers (manifest_headers/media_headers) are stored on the **profile** instead of per-channel. One token refresh updates all channels. Use for providers where all channels share the same account-level CDN token (DAZN, TataPlay, etc.) |
| `remux_segment_duration` | int | `6` | HLS output segment duration in seconds (internal_remux only) |
| `remux_playlist_window` | int | `10` | Number of segments in HLS playlist (internal_remux only) |
| `remux_playlist_duration` | int | `60` | Max HLS playlist duration in seconds (internal_remux only) |
| `remux_speed_up` | int | `1` | 1 = burst initial segments faster for quick startup |
| `live_edge_segments` | int | `0` | Skip to last N segments for live streams. 0 = play from start, 5-10 = near live edge |

### Example: DASH + Widevine Provider

```
# Profile defaults for DAZN-type provider (DASH + Widevine + proxy + expiring tokens)
profile = {
    "running_mode": "internal_remux",    # Best for DASH sources
    "output_format": "hls",              # Most compatible output
    "manifest_refresh_min": 5,           # Refresh CDN token every 5 min
    "proxy_from_provider": True,         # Copy proxy from provider config
    "shared_headers": 1,                 # All channels share same CDN auth
    "remux_segment_duration": 6,
    "remux_playlist_window": 10,
    "remux_playlist_duration": 60,
}
groups = set(ch["group"] for ch in channels if ch.get("group"))
profile_defaults = {g: profile for g in groups}

print(json.dumps({"channels": channels, "profile_defaults": profile_defaults}))
```

### Example: HLS Provider (no expiring tokens)

```
# Profile defaults for HLS provider with static tokens
profile = {
    "running_mode": "ffmpeg_remux",      # FFmpeg for HLS sources
    "output_format": "hls",
    "proxy_from_provider": True,
    "shared_headers": 1,
}
groups = set(ch["group"] for ch in channels if ch.get("group"))
profile_defaults = {g: profile for g in groups}

print(json.dumps({"channels": channels, "profile_defaults": profile_defaults}))
```

### Shared Headers

When `shared_headers: 1`, CDN authentication tokens from `manifest_headers` are stored on the **profile** instead of each individual channel. This means:

**One refresh benefits all channels** — when pre-start refresh runs for any channel, the fresh token is saved to the profile. All other channels using that profile immediately get the new token.

**Efficient token management** — instead of N refresh calls for N channels, one call is enough.

**Works with manifest_refresh_min** — periodic token refresh during playback also updates the profile.

Use `shared_headers: 1` for any provider where all channels use the same account-level CDN authentication (most providers).

### Updating Existing Profiles

Profile defaults are only applied when a profile is **first created**. To re-apply script defaults to existing profiles:

Open the Sync modal for the provider

Check **"Update profile settings from script"**

Click Sync — only script-defined values are updated, your manual customizations are preserved

## Logging

Streamora captures `stderr` and displays it in the terminal UI in real time. Use it for progress updates, debug info, and status messages. `stdout` is reserved exclusively for the JSON result.

### Logging Helper

```
import sys

def log(msg):
    """Print to stderr (shown in Streamora terminal)."""
    print(msg, file=sys.stderr, flush=True)

# Usage
log("Authenticating...")
log("Fetching channel list...")
log(f"  Found: ESPN HD (dash, drm=widevine)")
log(f"Total: 42 channels found")
```

### Rules

`flush=True` is important — without it, messages may be buffered and appear delayed or all at once.

Never print non-JSON to `stdout`. This causes parse errors: `invalid character 'L' looking for beginning of value`.

Python's default `print()` goes to stdout — always use `file=sys.stderr` for logs.

Libraries like `requests` may print warnings to stderr — this is fine, Streamora handles it.

### What Goes Where

| Output | Destination | Purpose |
|---|---|---|
| `print(json.dumps(result))` | stdout | JSON result (parsed by Streamora) |
| `print("...", file=sys.stderr)` | stderr | Progress/debug (shown in terminal UI) |
| `sys.exit(1)` | exit code | Signal error (non-zero = failure) |

## Authentication & Token Caching

Since scripts are stateless (each invocation is a new process), you need to handle authentication yourself. Most providers use OAuth/JWT tokens that can be cached to disk.

### Token Caching Pattern

```
import os
import json
import time

CACHE_DIR = os.path.dirname(os.path.abspath(__file__))
CACHE_FILE = os.path.join(CACHE_DIR, '.my_provider_cache.json')

def load_cache():
    """Load cached tokens from disk."""
    try:
        with open(CACHE_FILE, 'r') as f:
            return json.load(f)
    except (FileNotFoundError, json.JSONDecodeError):
        return {}

def save_cache(data):
    """Save tokens to disk."""
    with open(CACHE_FILE, 'w') as f:
        json.dump(data, f)

def get_token(username, password):
    """Get a valid token, using cache if possible."""
    cache = load_cache()
    # Check if cached token is still valid (e.g. 1 hour TTL)
    if cache.get('token') and cache.get('expires', 0) > time.time():
        log("Using cached token")
        return cache['token']

    # Authenticate and get new token
    log("Authenticating...")
    resp = requests.post('https://api.example.com/auth', json={
        'username': username, 'password': password
    })
    resp.raise_for_status()
    token = resp.json()['token']

    # Cache it
    save_cache({'token': token, 'expires': time.time() + 3500})
    return token
```

### Best Practices

- Cache tokens in the same directory as the script (use `__file__` to get the path).
- Use a dot-prefixed filename (e.g. `.my_cache.json`) to keep it hidden.
- Always include an expiry timestamp and validate before using cached tokens.
- Handle token refresh gracefully — if a cached token fails, re-authenticate and retry.
- Credentials come as CLI arguments, never hardcode them in the script.

## DRM Key Acquisition

Streamora has a built-in Widevine CDM that handles most DRM-protected streams automatically. Your script only needs to provide the `license_url`, `license_headers`, and `pssh` in the manifest response. The `action=cdm` is a fallback for special cases.

### When Is action=cdm Called?

- Channel has `drm_type` set to `widevine` or `playready`
- No `keys` returned in the manifest response
- Built-in CDM fails (e.g. license server requires special auth)
- Then Streamora calls `action=cdm` as a last resort

### PSSH Extraction from MPD

If your provider returns a DASH manifest, you can extract the PSSH from it:

```
import base64
import re
import requests

def extract_pssh(mpd_url, headers=None):
    """Extract Widevine PSSH from a DASH manifest."""
    resp = requests.get(mpd_url, headers=headers or {})
    # Look for Widevine system ID
    pssh_match = re.search(
        r'<cenc:pssh[^>]*>(.*?)</cenc:pssh>',
        resp.text
    )
    if pssh_match:
        return pssh_match.group(1).strip()
    return None
```

### Using pywidevine for CDM

```
from pywidevine.cdm import Cdm
from pywidevine.device import Device
from pywidevine.pssh import PSSH
import requests

def get_keys(pssh_b64, license_url, license_headers=None, wvd_path=None):
    """Acquire Widevine keys using pywidevine."""
    device = Device.load(wvd_path or '/root/.wvd/device.wvd')
    cdm = Cdm.from_device(device)
    session_id = cdm.open()

    pssh = PSSH(pssh_b64)
    challenge = cdm.get_license_challenge(session_id, pssh)

    resp = requests.post(
        license_url,
        data=challenge,
        headers=license_headers or {}
    )
    resp.raise_for_status()

    cdm.parse_license(session_id, resp.content)
    keys = []
    for key in cdm.get_keys(session_id):
        if key.type == 'CONTENT':
            keys.append(f"{key.kid.hex}:{key.key.hex()}")
    cdm.close(session_id)

    return '\n'.join(keys)
```

### ClearKey (No CDM Needed)

For ClearKey DRM, return the keys directly in the manifest response:

```
{
  "manifest_url": "https://example.com/stream.mpd",
  "drm_type": "clearkey",
```

```
  "keys": "aabbccdd11223344:11223344aabbccdd",
  "license_url": "",
  "pssh": ""
}
```

## Skeleton Script

Copy this starter template and fill in the provider-specific logic:

```
#!/usr/bin/env python3
"""my_provider.py - Streamora provider script template."""
import sys
import json
import requests

# ── Helpers ──────────────────────────────────────
def parse_arg(args, key):
    for arg in args:
        if arg.startswith(key + '='):
            return arg.split('=', 1)[1]
    return None

def log(msg):
    print(msg, file=sys.stderr, flush=True)

# ── Parse arguments ──────────────────────────────
action = parse_arg(sys.argv, 'action')
channel_id = parse_arg(sys.argv, 'id')
username = parse_arg(sys.argv, 'user')
password = parse_arg(sys.argv, 'password')

# ── action=channels ──────────────────────────────
if action == "channels":
    log("Authenticating...")
    # TODO: authenticate with provider API
    # token = login(username, password)

    log("Fetching channel list...")
    # TODO: fetch channels from provider
    channels = [
        {
            "id": "ch_001",
            "name": "Channel One",
            "source_url": "",
            "source_type": "dash",
            "drm_type": "widevine",
            "group": "General",
            "logo": "",
            "epg_id": ""
        }
    ]

    for ch in channels:
        log(f"  Found: {ch['name']} ({ch['source_type']}, drm={ch['drm_type']})")

    # Profile defaults — auto-configure profiles on first sync
    profile = {
        "running_mode": "internal_remux",
        "output_format": "hls",
        "shared_headers": 1,
        # "manifest_refresh_min": 5,    # Uncomment for expiring CDN tokens
        # "proxy_from_provider": True,  # Uncomment if proxy needed
    }
    groups = set(ch["group"] for ch in channels if ch.get("group"))
    profile_defaults = {g: profile for g in groups}

    log(f"Total: {len(channels)} channels")
    print(json.dumps({"channels": channels, "profile_defaults": profile_defaults}))

# ── action=manifest ──────────────────────────────
elif action == "manifest":
    if not channel_id:
        print(json.dumps({"error": "channel id required"}))
        sys.exit(1)

    log(f"Fetching manifest for: {channel_id}")
    # TODO: get stream URL and DRM info for this channel
    result = {
        "manifest_url": "https://cdn.example.com/stream.mpd",
```

```
        "manifest_headers": {},
        "license_url": "https://drm.example.com/license",
        "license_headers": {"Origin": "https://example.com"},
        "pssh": "",
        "drm_type": "widevine",
        "source_type": "dash",
        "keys": "",
        "error": ""
    }
    log(f"  URL: {result['manifest_url']}")
    log(f"  DRM: {result['drm_type']}")
    print(json.dumps(result))

# ── action=cdm (optional fallback) ──────────────
elif action == "cdm":
    if not channel_id:
        print(json.dumps({"error": "channel id required"}))
        sys.exit(1)

    log(f"Acquiring CDM keys for: {channel_id}")
    # TODO: custom DRM key acquisition logic
    # Only implement if built-in CDM doesn't work for this provider
    keys = ""  # "kid_hex:key_hex\nkid2_hex:key2_hex"
    print(json.dumps({"keys": keys, "error": ""}))

else:
    print(json.dumps({"error": f"unknown action: {action}"}))
    sys.exit(1)
```

## Reference: demo_provider.py

This annotated walkthrough of the built-in demo provider shows the complete protocol in action with test data. No external dependencies are required — it uses hardcoded responses to demonstrate all three actions.

### Full Script

```
#!/usr/bin/env python3
"""Demo provider script for Streamora.
Implements the 3-action protocol: channels, manifest, cdm.
Uses hardcoded test data - no external dependencies needed.
"""
import sys
import json
import time

def parse_arg(args, key):
    for arg in args:
        if arg.startswith(key + '='):
            return arg.split('=', 1)[1]
    return None

def log(msg):
    print(msg, file=sys.stderr, flush=True)

action = parse_arg(sys.argv, 'action')
channel_id = parse_arg(sys.argv, 'id')

# ─── Channel Data ──────────────────────────────────
CHANNELS = [
    {"id": "demo_sports", "name": "Demo Sports HD",
     "logo": "", "group": "Sports",
     "source_url": "", "source_type": "dash",
     "drm_type": "widevine", "epg_id": "demo.sports"},
    {"id": "demo_news", "name": "Demo News 24/7",
     "logo": "", "group": "News",
     "source_url": "http://example.com/news/stream.m3u8",
     "source_type": "hls", "drm_type": "none",
     "epg_id": "demo.news"},
    {"id": "demo_movies", "name": "Demo Movies",
     "logo": "", "group": "Entertainment",
     "source_url": "", "source_type": "dash",
     "drm_type": "clearkey", "epg_id": "demo.movies"},
]

# ─── Manifest Data ─────────────────────────────────
MANIFESTS = {
    "demo_sports": {
        "manifest_url": "https://example.com/dash/sports/manifest.mpd",
        "manifest_headers": {},
        "license_url": "https://drm.example.com/license/sports",
        "license_headers": {"Origin": "https://example.com"},
        "pssh": "AAAAOHBzc2gAAAAA7e+LqXnWSs6jyCfc1R0h7Q...",
        "drm_type": "widevine", "source_type": "dash",
        "keys": "", "error": ""
    },
    "demo_movies": {
        "manifest_url": "https://example.com/dash/movies/manifest.mpd",
        "manifest_headers": {},
        "license_url": "", "license_headers": {},
        "pssh": "", "drm_type": "clearkey", "source_type": "dash",
        "keys": "aabbccdd11223344:11223344aabbccdd",  # Direct ClearKey
        "error": ""
    },
}

# ─── CDM Keys ──────────────────────────────────────
CDM_KEYS = {
    "demo_sports": "aabb1122aabb1122:1122aabb1122aabb\n"
                   "ccdd3344ccdd3344:3344ccdd3344ccdd",
}

# ═══════════════════════════════════════════════════
```

```
if action == "channels":
    log("Connecting to demo provider API...")
    time.sleep(0.3)  # Simulate network delay
    log("Authenticated successfully")

    log("Fetching channel list...")
    for ch in CHANNELS:
        log(f"  Found: {ch['name']} ({ch['source_type']}, "
            f"drm={ch['drm_type']})")

    log(f"Total: {len(CHANNELS)} channels found")
    # THE JSON RESULT - only this goes to stdout
    print(json.dumps({"channels": CHANNELS}))

elif action == "manifest":
    if not channel_id:
        print(json.dumps({"error": "channel id required"}))
        sys.exit(1)

    log(f"Fetching manifest for: {channel_id}")
    manifest = MANIFESTS.get(channel_id)
    if manifest:
        log(f"  URL: {manifest['manifest_url']}")
        log(f"  DRM: {manifest['drm_type']}")
        if manifest.get('keys'):
            log("  ClearKey: keys provided directly")
        print(json.dumps(manifest))
    else:
        print(json.dumps({"error": f"channel {channel_id} not found"}))
        sys.exit(1)

elif action == "cdm":
    if not channel_id:
        print(json.dumps({"error": "channel id required"}))
        sys.exit(1)

    log(f"Requesting CDM keys for: {channel_id}")
    keys = CDM_KEYS.get(channel_id)
    if keys:
        count = len(keys.strip().split('\n'))
        log(f"  Acquired {count} key(s)")
        print(json.dumps({"keys": keys, "error": ""}))
    else:
        log(f"  No CDM keys available for {channel_id}")
        print(json.dumps({"keys": "",
            "error": f"no CDM handler for {channel_id}"}))

else:
    log(f"Unknown action: {action}")
    print(json.dumps({"error": f"unknown action: {action}"}))
    sys.exit(1)
```

### Key Takeaways

**Separation of concerns:** All `log()` calls go to stderr, only `json.dumps()` goes to stdout.

**Graceful errors:** Missing channels return JSON with an `"error"` field plus `sys.exit(1)`.

**ClearKey shortcut:** `demo_movies` returns keys directly in the manifest, skipping the CDM step.

**CDM fallback:** `demo_sports` returns keys via `action=cdm` when the built-in CDM can't handle it.

**Simulated delays:** `time.sleep()` mimics real API calls. Remove these in production scripts.

### Testing Locally

```
# Test channels action
python3 /root/scripts/demo_provider.py action=channels

# Test manifest for a specific channel
python3 /root/scripts/demo_provider.py action=manifest id=demo_sports

# Test CDM fallback
python3 /root/scripts/demo_provider.py action=cdm id=demo_sports
```

## Troubleshooting

### MODULENOTFOUNDERROR: NO MODULE NAMED 'XYZ'

A Python package is missing. Install it on the server:

```
pip3 install xyz
```

Common packages: `requests`, `curl_cffi`, `xmltodict`, `PyJWT`, `pycryptodome`, `protobuf`, `pywidevine`

### PARSE CHANNELS OUTPUT: INVALID CHARACTER...

The script printed non-JSON to stdout. This usually means `print()` calls that should use `file=sys.stderr`. Check for:

- Debug `print()` statements without `file=sys.stderr`
- Library code that prints warnings to stdout
- Exception tracebacks (these go to stderr by default, which is fine)

### SCRIPT EXITED: EXIT STATUS 1

The script crashed. Check the terminal log for the Python traceback. Common causes:

- Missing or wrong credentials
- API rate limiting or geo-blocking
- Network errors (timeout, connection refused)
- Invalid JSON response from the provider API

### NO CHANNELS FOUND / EMPTY SYNC

Verify credentials are correct and test manually:

```
cd /root/scripts && python3 my_script.py action=channels user=you@email.com password=secret
```

### KEYS NOT REFRESHING / DRM ERRORS

- Check that `license_url` and `pssh` are returned in the manifest response
- Verify `license_headers` include required auth headers (Origin, tokens, etc.)
- Try the CDM step manually: `python3 script.py action=cdm id=ch_id ...`
- Check if the license server requires a specific User-Agent or Origin header

### PIP3: COMMAND NOT FOUND

```
apt-get update && apt-get install -y python3-pip
```

### SCRIPT WORKS IN SSH BUT FAILS IN STREAMORA

- Working directory is set to the script's folder — use relative paths or `os.path.dirname(__file__)`
- Environment variables may differ — don't rely on shell env vars
- Make sure credentials are passed via the provider config, not hardcoded

## Provider API Endpoints

REST endpoints for managing providers programmatically. All require JWT authentication.

### Provider Management

| Method | Endpoint | Description |
|---|---|---|
| `GET` | `/api/providers` | List all providers |
| `POST` | `/api/providers` | Create a new provider |
| `GET` | `/api/providers/{id}` | Get provider details |
| `PUT` | `/api/providers/{id}` | Update provider configuration |
| `DELETE` | `/api/providers/{id}` | Delete provider (channels remain) |

### Execution

| Method | Endpoint | Description |
|---|---|---|
| `POST` | `/api/providers/{id}/sync` | Trigger channel sync |
| `POST` | `/api/providers/{id}/refresh-keys` | Refresh DRM keys for all channels |
| `POST` | `/api/providers/{id}/stop` | Stop a running execution |
| `POST` | `/api/providers/{id}/test` | Test provider (same as sync) |

### Results & Scripts

| Method | Endpoint | Description |
|---|---|---|
| `GET` | `/api/providers/{id}/runs` | List execution history (last 50) |
| `GET` | `/api/providers/{id}/runs/{runId}` | Get full log for a run |
| `GET` | `/api/providers/{id}/channels` | List channels linked to provider |
| `GET` | `/api/providers/scripts` | List available .py scripts |
| `POST` | `/api/providers/upload-script` | Upload a script (multipart/form-data) |

### Provider Object

```
{
  "id": 1,
  "name": "My Provider",
  "type": "script",
  "script_path": "/root/scripts/my_provider.py",
  "credentials": "{\"user\": \"john\", \"password\": \"secret\"}",
  "config": "{\"proxy\": \"http://proxy:8080\"}",
  "profile_name": "Sports",
  "sync_interval": 60,
  "key_refresh_interval": 720,
  "last_sync": "2026-03-04T16:30:00Z",
  "last_key_refresh": "2026-03-04T14:00:00Z",
  "status": "idle",
  "channel_count": 125,
  "enabled": 1,
  "total_runs": 45,
  "successful_runs": 43,
  "failed_runs": 2
}
```

### Script Locations

Streamora scans these directories for `.py` scripts:

- `/root/scripts/` — Primary location (uploads go here)
- `/opt/scripts/` — Alternative

`/home/streamora/scripts/` — Alternative